Estimating π Using Probability

A reproducible Monte Carlo simulation study

Student Research Report

Student: [Your name]
Class: XII ISC
Date: [Add submission date]

Research question: How does increasing the number of uniformly random points affect the accuracy of a Monte Carlo estimate of π?

Important: This is a classic method, not a claim of a new discovery. The study focuses on a reproducible experiment comparing sample sizes and error measures.

1. Abstract

This project investigates a probability-based method for approximating π. Random points are sampled uniformly from a square containing a unit circle, and the proportion inside the circle is used to estimate π. For each of four sample sizes (100, 1,000, 10,000, 100,000), 30 independent pseudo-random runs were performed using recorded seeds. The results show how estimates and error measures vary with sample size. The experiment is reproducible because the random seeds and individual results are saved in a CSV file.

2. Introduction

Monte Carlo methods use repeated random sampling to estimate quantities that may be difficult to calculate directly. In this example, the geometry of a circle turns the estimation of π into an estimation of a probability. The experiment is useful for learning about simulation, random variation, the law of large numbers, and the difference between a single result and a pattern across repeated trials.

3. Mathematical model

Consider a square with coordinates $-1 \le x \le 1$ and $-1 \le y \le 1$. Its area is 4. A circle of radius 1 centred at the origin has area π. If points are sampled uniformly, the probability that a point lies inside the circle is the ratio of the two areas:

$$P(\text{inside}) = \text{area of circle} / \text{area of square} = \pi / 4$$

A point is inside or on the circle when $x^2 + y^2 \le 1$. If N points are sampled and K lie inside, then K/N estimates π/4. Therefore the Monte Carlo estimator is $\pi\blacksquare = 4K/N$. As N increases, the sample proportion tends to become more stable, but a larger sample does not guarantee that every individual estimate is closer to π than the previous estimate.

4. Research question and hypothesis

Question: How does sample size affect the error of the estimate?
Hypothesis: Across repeated runs, the typical error scale should decrease as sample size increases. Sampling error is expected to decrease approximately in proportion to $1/\sqrt{N}$, rather than 1/N.

5. Methodology

The experiment used Python's standard random-number generator. For each N = 100, 1,000, 10,000, 100,000, 30 runs were performed. A fixed master seed (20261010) was used to derive a distinct seed for every run. For each generated point, x and y were selected uniformly from –1 to 1. The program counted points satisfying $x^2 + y^2 \le 1$ and calculated $\pi\blacksquare = 4K/N$. For each sample size, the report records the mean estimate, standard deviation, mean absolute error, root mean square error (RMSE), and theoretical standard error.

6. Results

N	Mean estimate	Mean abs. error	RMSE	Theoretical SE
100	3.146667	0.128000	0.153388	0.164218
1,000	3.134000	0.033091	0.038405	0.051930
10,000	3.140733	0.011640	0.014522	0.016422
100,000	3.142349	0.004487	0.005511	0.005193

The values above are computed from the 120 simulation runs generated for this report; they are not manually invented. Your own rerun may differ if you change the seed, code, or experiment settings.

7. Graphs

Figure 1. Estimate of π at increasing checkpoints in one reproducible run.

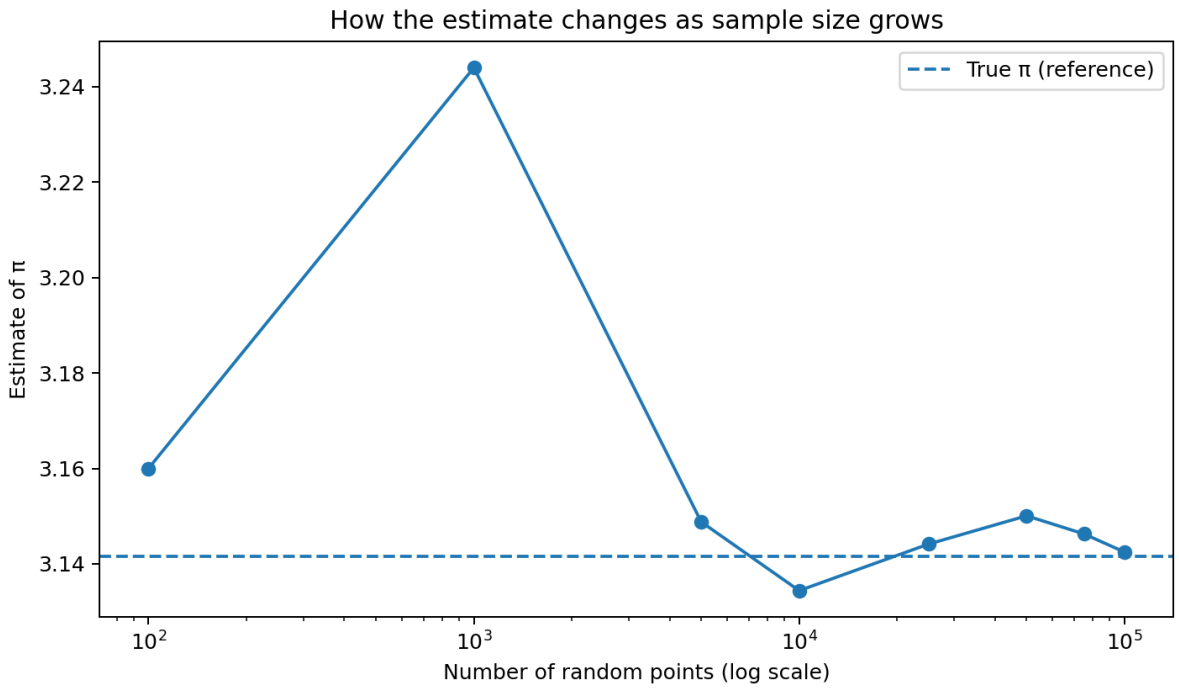
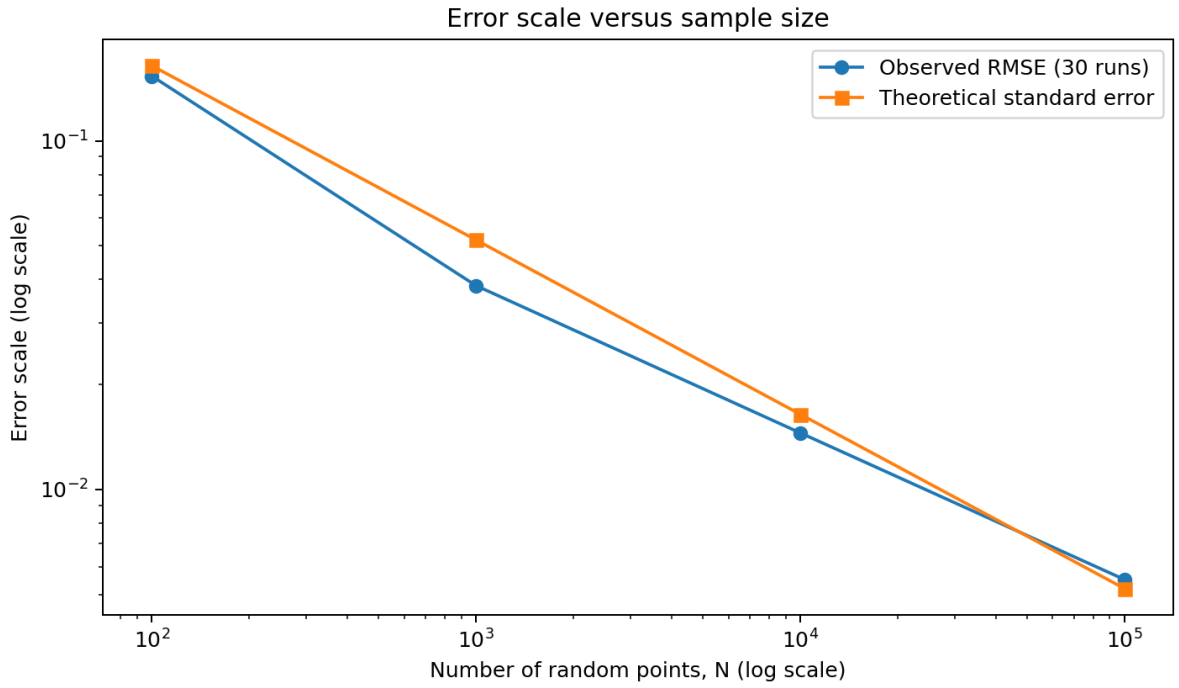


Figure 2. Observed RMSE compared with the theoretical standard error across 30 runs per sample size.



8. Discussion

Interpret the table and graphs rather than assuming the hypothesis must be true in every row. The mean estimate across repeated runs is expected to be near π, while individual estimates vary. The RMSE summarizes the typical error size across runs and can be compared with the theoretical standard error. With only 30 repetitions per sample size, the observed RMSE itself is subject to sampling variation. A useful extension is to increase the number of repetitions or compare different sampling methods.

9. Limitations

This is a simulation, not a physical experiment. The random numbers are pseudo-random, and the study uses only four sample sizes and 30 repetitions each. The method converges slowly compared with specialised numerical algorithms for computing π. Because π is already known to high precision, the exercise demonstrates probability and numerical error rather than providing a practical way to calculate π. The study should not be presented as original research in the sense of a new mathematical discovery.

10. Conclusion

The experiment provides a reproducible demonstration of how random sampling can estimate a geometric quantity. The proportion of points inside the unit circle estimates π/4, leading to π■ = 4K/N. Repeated runs make it possible to examine error more responsibly than relying on a single simulation. The next step is for the student to rerun the code, explain the actual results in their own words, and document any additional experiment they personally design.

11. References

• Carnegie Mellon University, Principles of Computing, “The Monte Carlo Method and π.” https://www.cs.cmu.edu/~tcortina/15110f11/lab9/index.html

• McGill University, Teaching and Learning Services, “Calculating π.” https://tsi-mcgill.github.io/tsi-tutorials/Python/Calculating_pi/

• University of Wisconsin–Madison, STAT 340, “Monte Carlo.” https://pages.stat.wisc.edu/~kdlevin/teaching/Fall2022/STAT340/lecs/L02_MonteCarlo.html

• Python Software Foundation, Python documentation: random — Generate pseudo-random numbers. https://docs.python.org/3/library/random.html

• Matplotlib documentation. https://matplotlib.org/stable/

Appendix: Code and raw data

The complete Python source is in pi_monte_carlo.py. Individual run-level data are in replicate_results.csv, and aggregate statistics are in summary_results.csv. These files allow the calculations to be checked and reproduced.